

Vulnerability, Discovery and Exploitation (VDE)

Coursework 2

REPORT

Student No.: 2196059

Word Count: 1455

Disclaimer

By submitting this report for marking, **I fully understand** that and **I agree**:

- This is an independent piece of coursework, and it is expected that I have taken responsibility for all the design, implementation, analysis of results and writing of the report.
- For the Turnitin plagiarism software, the match score of the report must be below 15% overall and less than 5% to an individual source (excluding appendices). Marks will be hand checked and investigated that exceed the 15% margin.
- This marking scheme sets out what would be expected for each marking band for this module's coursework.
- It is blind marked.
- Any technical help, high-level advice and suggestions which may be provided in-person (e.g., labs) and electronically, has no contribution to or an indication of my final mark.
- Knowledge of a peer's work and their final mark is not justification for what my own mark should be.
- Any examples provided were used for ideas only, and "having followed an example" is not justification for what my mark should be.

Word Count

Section	Word Count
Heap Overflow Vulnerability	150
Memory Allocation Influenced by Lack of Boundaries	153
Failure to Guarantee Deallocation	151
Format String Injection	189
Stack Buffer Overflow	151
Format String Vulnerability	195
Heap Buffer Overflow	141
Command Injection Vulnerability	155
Filesize Taken Directly From External Input	170
Total	1455

Vulnerability Analysis - file1.c

Heap Overflow Vulnerability

Due to the way **file1.c** allocates the memory for **nItems**, **MAX_ITEMS** controls the data that is actually written into the program.

```
DataItem *items = malloc(allocSize);
```

This part of the code defines the maximum capacity of the array within the memory. However:

```
while (count < MAX_ITEMS && fscanf(fp, "%63s", stackbuf) == 1)
```

The while loop used does **not** check whether the count is above **nItems**. Because of the way **file1.c** reads **data1.txt** to allocate size, **size_t allocSize = nItems * sizeof(DataItem);** - If **data1.txt** starts with the number **1**, then **nItems** will equal **1** and **allocSize** will only allocate the space for **1 DataItem**.

However, because **MAX_ITEMS** can be larger than **allocSize** and the loop keeps the count going based off of **MAX_ITEMS**, the count can go above the allocated size and then cause a **Heap Buffer Overflow** because **file1.c** will then start to write over the allocated heap block.

```
Breakpoint 1, processFile (filename=0x8048b90 "data1.txt", itemCount=0xffffd0f8) at file1.c:51
warning: Source file is more recent than executable.
51      FILE *fp = fopen(filename, "r");
(gdb) step
52      if (!fp) {
(gdb) step
58      fscanf(fp, "%u", &nItems);
(gdb) step 100
ste86      count++;
(gdb) step 100
76      if (sscanf(stackbuf, "%d", &num) == 1) {
(gdb) print count
51 = 24
(gdb) print nItems
52 = 1
(gdb) print sizeof(DataItem)
53 = 12
```

```

0x804c2f0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c300: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c310: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c320: 0x00000000 0x00000000 0x00000000 0x00000000 0x00020cd9
(gdb) step 10
73         if (!items[count].word) continue;
(gdb) print count
$22 = 2
(gdb) print nItems
$23 = 1
(gdb) x/24x items
0x804c2d0: 0x0804c2e0 0x00000000 0x00000000 0x00000000 0x0804c330
0x804c2e0: 0x00000020 0x00000001 0x0804c380 0x00000000 0x00000000
0x804c2f0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c300: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c310: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804c320: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000051
(gdb) step 100
84         strcpy(items[count].word, stackbuf);
(gdb) print count
$24 = 14
(gdb) print nItems
$25 = 1
(gdb) x/24x items
0x804c2d0: 0x0804c2e0 0x00000000 0x00000000 0x00000000 0x0804c330
0x804c2e0: 0x00000020 0x00000001 0x0804c380 0x00000000 0x00000000
0x804c2f0: 0x00000000 0x0804c3d0 0x00000020 0x00000001 0x00000000
0x804c300: 0x0804c420 0x00000000 0x00000000 0x0804c470 0x00000000
0x804c310: 0x00000020 0x00000001 0x0804c4c0 0x00000000 0x00000000
0x804c320: 0x00000000 0x0804c510 0x00000020 0x00000001 0x00000000

```

```

Breakpoint 1, processFile (filename=0x8048b90 "data1.txt", itemCount=0xffffd0f8) at file1.c:70
70     while (count < MAX_ITEMS && fscanf(fp, "%63s", stackbuf) == 1) {
(gdb) x/24x stackbuf
0xffffd07c: 0xffffd130 0xf7fb7000 0x00000000 0xffffd118
0xffffd08c: 0xf7e140b9 0xffffd3ba 0x00000000 0x0000000a
0xffffd09c: 0x00000000 0xf7fb7c40 0xf7ded454 0xf7ffd940
0xffffd0ac: 0x000000c2 0xf7fe4f09 0xffffd130 0xf7e14095
0xffffd0bc: 0x00000001 0x0804c2d0 0x0000000c 0x0804b160
0xffffd0cc: 0x00000000 0xffffd3b8 0xffffd130 0xffffd118
(gdb) step 50
73         if (!items[count].word) continue;
(gdb) x/24x stackbuf
0xffffd07c: 0x74003939 0x43420000 0x42434242 0x43424243
0xffffd08c: 0x43424341 0x43424142 0x42434241 0x41424341
0xffffd09c: 0x42414342 0x43414243 0x43424142 0x42434142
0xffffd0ac: 0x41434241 0x42414342 0x43420043 0x00434242
0xffffd0bc: 0x00000001 0x0804c2d0 0x0000000c 0x0804b160
0xffffd0cc: 0x00000006 0xffffd3b8 0xffffd130 0xffffd118
(gdb) step
76         if (sscanf(stackbuf, "%d", &num) == 1) {
(gdb)
77             items[count].number = num;
(gdb)
78             items[count].isNumber = 1;
(gdb)
79             sprintf(items[count].word, "%d", num);
(gdb)
86             count++;
(gdb)
70     while (count < MAX_ITEMS && fscanf(fp, "%63s", stackbuf) == 1) {
(gdb)
89     fclose(fp);
(gdb)
corrupted size vs. prev_size

Program received signal SIGABRT, Aborted.
0xf7fd4df9 in __kernel_vsyscall ()

```

Memory Allocation Influenced by Lack of Boundaries

Using the original **data1.txt** that starts with the string **AAAA**, **file1.c** attempts to read the number of items that have been inputted through the file via the following sequence:

```
fscanf(fp, "%u", &nItems)
```

The function **fscanf()** is used to read formatted data from a file, writing it into a memory location specified by the arguments following it then moving the position to where the file stopped reading.

However, **file1.c** then uses the following to determine the size allocated for the items:

```
size_t allocSize = nItems * sizeof(DataItem)
```

But as shown below, **fscanf** does not actually contain any boundaries to ensure the memory allocated is valid. An attacker could also utilize this vulnerability to help shape the incorrect memory allocation via how **fscanf()** processes the input which can then influence how the data is parsed into **nItems**, where **nItems** then influences the size allocated in **allocSize**, allowing the attacker to exploit this.

```
Breakpoint 1, processFile (filename=0x8048b90 "data1.txt",
    itemCount=0xffffd0f8) at file1.c:59
warning: Source file is more recent than executable.
59      size_t allocSize = nItems * sizeof(DataItem);
(gdb) print nItems
$1 = 6
(gdb) print allocSize
$2 = 0
(gdb) step
61      DataItem *items = malloc(allocSize);
(gdb) print allocSize
$3 = 72
(gdb)
```

```
Breakpoint 1, processFile (filename=0x8048b90 "data1.txt",
    itemCount=0xffffd0f8) at file1.c:59
59      size_t allocSize = nItems * sizeof(DataItem);
(gdb) step
61      DataItem *items = malloc(allocSize);
(gdb) print nItems
$1 = 4158720240
(gdb) print allocSize
$2 = 2660002624
(gdb)
```

Failure to Guarantee Deallocation

Because of **file1.c** dynamically allocating memory for every single **DataItem** within the code and the usage of **malloc** in the string:

```
items[count].word = malloc(MAX_BUFFER);
```

This is caused due to how **file1.c** expects to use the **FreeItems()** function at the end of the code. The structure of the code however makes it possible for the function to not be reached at all within the run of **file1.c**, for example if the program is terminated early or encounters an error which causes a crash, then the allocated heap memory via **malloc(MAX_BUFFER)**; is not released.

As shown in the second figure below, valid addresses are assigned to the items inputted from **data1.txt** and assigned to **items[i].word**, proving that successful allocation is being carried out during execution, but due to the lack of guarantee that the **FreeItems()** function will run, **free()** may not execute at all.

```
106     }
107     }
108     return NULL;
109 }
110
(gdb)
111 void freeItems(DataItem *items, int itemCount) {
112     if (!items) return;
113     for (int i = 0; i < itemCount; i++) {
114         free(items[i].word);
115     }
116     free(items);
117 }
118

(gdb) step
70     while (count < MAX_ITEMS && fscanf(fp, "%63s", stackbuf) == 1) {
(gdb) step
72         items[count].word = malloc(MAX_BUFFER);
(gdb) step
73         if (!items[count].word) continue;
(gdb) print items[count].word
$1 = 0x804c2d0 ""
(gdb) print items[0].word
$2 = 0x804c2d0 ""
(gdb) step 10
77         items[count].number = num;
(gdb) step 10
84         strcpy(items[count].word, stackbuf);
(gdb) print items[0].word
$3 = 0x804c2d0 "AAAA"
(gdb) print items[1].word
$4 = 0x804c320 "42"
(gdb)
```

Format String Injection

Because of **File1.c's** use of both user input and **printf** without any input validation the insecure design of the **printf()** function allows an attacker to carry out Format String Injection.

```
if (argc > 1) index = atoi(argv[1]);  
if (argc > 2) type = atoi(argv[2]);
```

```
printf("Requested index: ");  
printf(argv[1] ? argv[1] : "0");  
printf("\n");
```

An attacker could exploit this vulnerability to crash the program, which would cause a denial of service, or they could exploit the vulnerability to view and extract data that is stored within the process, for example, accessing data within the stack frame of a function through the use of **"%x"** or **"%p"** to print **unsigned ints** and **addresses or pointers**.

Furthermore, this vulnerability can be used with **"%n"** to overwrite the memory with arbitrary data, where the format string would overwrite the targeted location in the process' memory.

Below the format string injection vulnerability was exploited to print values and pointers in the memory through the use of **%x** and **%p**. Additionally, **%s** can be used to cause a denial of service as it makes the **printf()** function attempt to read invalid memory.

```
vde@vde:~/Documents/Coursework/Task1$ ./file1.exe "%x %x %x"  
Requested index: ff9ef7e8 0 f7d4339b  
Result: AAAA  
vde@vde:~/Documents/Coursework/Task1$ ./file1.exe "%p %p %p %p %p %p"  
Requested index: 0xffdb9d88 (nil) 0xf7d8d39b 0xf7f323fc (nil) 0x6  
Result: AAAA  
vde@vde:~/Documents/Coursework/Task1$ ./file1.exe "AAAAAAAAAA%n"  
Requested index: AAAAAAAAAA  
Result: AAAA  
vde@vde:~/Documents/Coursework/Task1$
```

```
vde@vde:~/Documents/Coursework/Task1$ ./file1.exe %s%s%s%s%s  
Segmentation fault (core dumped)
```

Vulnerability Analysis - file2.c

Stack Buffer Overflow

File2.c has a stack buffer overflow vulnerability in its code which is caused by the use of **fscanf** alongside the **%s** format specified in the following code:

```
char Format[8];
fscanf(inFile, "%s", Format);
```

Due to the lack of length restriction on the **%s** specifier it will read the input provided from **Data2.txt** to the point where the string will overflow the allocated stack space and after exceeding the size of the buffer, which is 8 bytes. Because of this vulnerability caused by a lack of proper input restrictions, the stack buffer overflow could lead to memory corruption, crashes or potential exploitation from attackers via shellcode injection.

This stack buffer overflow could also be used to exploit the return address by an attacker to access another function, an example being **do_command**, which would grant the attacker direct access to the shell to execute additional commands to compromise the program.

[illegible]

```
Starting program: /home/vde/Documents/Coursework/task2/file2.exe data2.txt
File size = 108
```

```
Breakpoint 1, readFile (argv=0xffffd3b3 "data2.txt") at file2.c:48
```

```
48     file=malloc(fileSize*sizeof(char));
```

```
(gdb) step
```

```
50     rewind(inFile);
```

```
(gdb)
```

```
52     fscanf(inFile,"%d",&ChunkID);
```

```
(gdb)
```

```
53     fscanf(inFile,"%d",&ChunkSize);
```

```
(gdb)
```

```
54     fscanf(inFile,"%s",Format);
```

```
(gdb)
```

```
55     fscanf(inFile,"%d",&bitsPerSample);
```

```
(gdb) print Format
```

```
$1 = "ABABABAB"
```

```
(gdb) print &Format
```

```
$2 = (char (*)[8]) 0xffffd0a4
```

```
(gdb) x/24x &Format
```

0xffffd0a4:	0x42414241	0x42414241	0x42414241	0x42414241
0xffffd0b4:	0x41414241	0x41424142	0x41424142	0x41424142
0xffffd0c4:	0x41424142	0x41424142	0x41424142	0x41424142
0xffffd0d4:	0x41424142	0x41424142	0x41424142	0x41424142
0xffffd0e4:	0x41424142	0xff004142	0x080486cd	0xffffd3b3
0xffffd0f4:	0x00000000	0xffffd1d0	0x0804896b	0x00000002

```
(gdb) step
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
warning: Source file is more recent than executable.
```

```
54     fscanf(inFile,"%s",Format);
```

```
(gdb) x/24x $esp
```

0xffffcd10:	0xffffcd84	0xf7fd379e	0x0804821c	0xffffcd84
0xffffcd20:	0xf7ffdb8c	0x00000001	0xf7fc1720	0x00000001
0xffffcd30:	0x00000000	0x6c796170	0x3164616f	0x7478742e
0xffffcd40:	0x00000000	0x00000000	0x00000000	0x0804e700
0xffffcd50:	0xffffcfe8	0x0804d1a0	0x0000004c	0xffffdfc0
0xffffcd60:	0xf7fc7570	0xf7fc7000	0xffffcd98	0x0804925d

```
(gdb) step
```

```
55     fscanf(inFile,"%d",&bitsPerSample);
```

```
(gdb) x/24x $esp
```

0xffffcd10:	0xffffcd84	0xf7fd379e	0x0804821c	0xffffcd84
0xffffcd20:	0xf7ffdb8c	0x41414141	0x41414141	0x41414141
0xffffcd30:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffcd40:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffcd50:	0x41414141	0x41414141	0x41414141	0x41414141
0xffffcd60:	0x41414141	0x41414141	0x41414141	0x08049216

```
(gdb) c
```

```
Continuing.
```

```
Program received signal SIGSEGV, Segmentation fault.
```

Format String Vulnerability

File2.c also contains a format string vulnerability because of its use of **printf** alongside user input and a lack of input validation as displayed in the code below:

```
strcpy(output,input);  
printf(output);
```

As the output variable is taken directly from file input through **Data2.txt** without any input sanitisation before **file2.c** prints it using **printf**, any of the format specifiers within **Data2.txt** are interpreted as instructions, which in this case I used **%x** to display the vulnerability. This causes values from the stack to be printed rather than the original string, which signifies the code is reading from unintended memory locations.

What this vulnerability can do is lead to data leakage from within **file2.c** by exposing sensitive content within the memory, or with the use of other format specifiers such as **%n** an attacker could access and write onto vulnerable memory locations.

Shown below, **%x** and **%p** are used to print out unsigned ints and addresses / pointers within the stack frame. Additionally, the repeated input of **%s** can be exploited to prompt a program crash to cause denial of service due to the **printf()** Function's attempts to read invalid memory.

```
Reading symbols from /home/vde/.local/bin/gdb...  
(gdb) run data2.txt  
Starting program: /home/vde/Documents/Coursework/task2/file2.exe data2.txt  
File size = 31  
Chunk ID = 5  
Chunk Size = 8  
Format = Base36  
Bits per sample = 4  
ID = 22  
804c6e0 ffffd0e8 f7e32d86 f7fb7d80 8048a2a ffffd084  
ID[Inferior 1 (process 3523) exited normally]  
(gdb)
```

```
5  
8  
Base36  
4  
%x %x %x %x %x %x  
ID22  
~  
~  
~  
~
```

```
vde@vde:~/Documents/Coursework/task2$ ./file2.exe data2.txt  
File size = 28  
Chunk ID = 5  
Chunk Size = 8  
Format = Base36  
Bits per sample = 4  
ID = 22  
0x860a6e00xffd5d2d80xf7d72d860xf7ef7d800x8048a2a0xffd5d2740x86092c0%
```

```
IDvde@vde:~/Documents/Coursework/task2$ vim data2.txt
vde@vde:~/Documents/Coursework/task2$ ./file2.exe data2.txt
```

File size = 49

Chunk ID = 5

Chunk Size = 8

Format = Base36

Bits per sample = 4

ID = D22

%S

`I(ffffS0?*)`

ID = %s

1%5%5%5%5%5%5%5%5%5%5%5%5%5%5%5%5%5%5%5%

[illegible]

Segmentation fault (core dumped)

```
vde@vde:~/Documents/Coursework/task2$
```

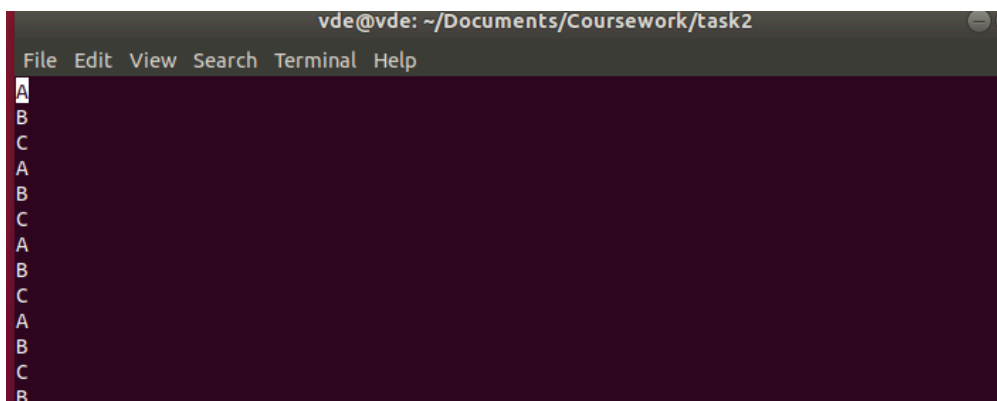
Heap Buffer Overflow

Due to the unsafe use of **malloc()** and **strcpy()** within the **display()** function of the program, the function allocates memory using the value of **filesize**:

```
int display(char *input, int filesize)
{
    char *output;
    output = malloc(filesize * sizeof(char));
    strcpy(output, input);
    printf(output);
}
```

Because of **strcpy()** having no set boundaries, it copies the bytes from the **input** pointer without verifying for things such as the length of **input** exceeding that of **filesize**, which will then allow an attacker to exploit the vulnerability by using the copy operation to inject data that will overflow from the allocated heap.

One of the ways this can be used is to cause a crash which leads to a denial of service from the program. Additionally, this could be done to overwrite the nearby data to execute functions outside of the allocated space for the heap.



```
gdb) x/x &output
0xffffccec: 0x00000000
gdb) x/24x 0x0804d2e0
0x0804d2e0: 0xf7f8c798 0xf7f8c798 0x00000000 0x00000000
0x0804d2f0: 0x41414141 0x41414141 0x41414141 0x41414141
0x0804d300: 0x41414141 0x41414141 0x41414141 0x3244490a
0x0804d310: 0x00000a32 0x00000000 0x00000000 0x00000000
0x0804d320: 0x00000000 0x00000000 0x00000000 0x00000000
0x0804d330: 0x00000000 0x00000000 0x00000000 0x00000000
gdb) step
8      strcpy(output,input);
gdb) x/24x 0x0804d2e0
0x0804d2e0: 0xf7f8caa8 0xf7f8caa8 0x0804d2d8 0x0804d2d8
0x0804d2f0: 0x41414141 0x41414141 0x41414141 0x41414141
0x0804d300: 0x41414141 0x41414141 0x41414141 0x00000fe1
```

```

Breakpoint 1, display (input=0x804e700 "\nC\nA\nB\nC\nA\nB\nC\nC\nA\nB\nB\nA\nB\nC\nA\nB\n", filesize=4
warning: Source file is more recent than executable.
77     output=malloc(filesize*sizeof(char));
(gdb) step
78     strcpy(output,input);
(gdb) x/24x 0x0804d2e0
0x804d2e0:    0xf7f8caa8    0xf7f8caa8    0x0804d2d8    0x0804d2d8
0x804d2f0:    0x0a430a43    0x0a420a41    0x0a410a42    0x0a430a42
0x804d300:    0x0a420a41    0x0a410a43    0x0a430a42    0x00000fe1
0x804d310:    0xf7f8c798    0xf7f8c798    0x00000000    0x00000000
0x804d320:    0x0a420a41    0x0a420a41    0x0a420a41    0x0a410a43
0x804d330:    0x0a410a42    0x00000a43    0x00000000    0x00000000
(gdb)

```

```

10 = 022

Breakpoint 1, display (input=0x804c6e0 'A' <repeats 19 times>, "\n\nI",
    filesize=32) at file2.c:77
77     output=malloc(filesize*sizeof(char));
(gdb) x/24x $esp
0xffffd060:    0xf7fb7d80    0x08048a2a    0xffffd084    0x00000000
0xffffd070:    0xf7fb7000    0x00000000    0xffffd0e8    0x080488d9
0xffffd080:    0x0804c6e0    0x00000020    0xffffd098    0xffffd0ec
0xffffd090:    0xf7ffda9c    0x0a000001    0x00323244    0x00000001
0xffffd0a0:    0x00000004    0x65736142    0xf7003633    0x00000008
0xffffd0b0:    0x00000005    0x61746164    0x78742e32    0x00000074
(gdb) step
78     strcpy(output,input);
(gdb) x/24x $esp
0xffffd060:    0xf7fb7d80    0x08048a2a    0xffffd084    0x0804b2c0
0xffffd070:    0xf7fb7000    0x00000000    0xffffd0e8    0x080488d9
0xffffd080:    0x0804c6e0    0x00000020    0xffffd098    0xffffd0ec
0xffffd090:    0xf7ffda9c    0x0a000001    0x00323244    0x00000001
0xffffd0a0:    0x00000004    0x65736142    0xf7003633    0x00000008
0xffffd0b0:    0x00000005    0x61746164    0x78742e32    0x00000074
(gdb)

```

```

77     output=malloc(filesize*sizeof(char));
(gdb) x/24x $esp
0xffffd060:    0xf7fb7d80    0x08048a2a    0xffffd084    0x00000000
0xffffd070:    0xf7fb7000    0x00000000    0xffffd0e8    0x080488d9
0xffffd080:    0x42434142    0x0000002c    0xffffd098    0xffffd0ec
0xffffd090:    0xf7ffda9c    0x00000001    0xf7fcf410    0x00000001
0xffffd0a0:    0x00000000    0x44434241    0x42434241    0x43424341
0xffffd0b0:    0x42434241    0x41424341    0x43414243    0x42434142
(gdb) step
78     strcpy(output,input);
(gdb) x/24x $esp
0xffffd060:    0xf7fb7d80    0x08048a2a    0xffffd084    0x0804b2c0
0xffffd070:    0xf7fb7000    0x00000000    0xffffd0e8    0x080488d9
0xffffd080:    0x42434142    0x0000002c    0xffffd098    0xffffd0ec
0xffffd090:    0xf7ffda9c    0x00000001    0xf7fcf410    0x00000001
0xffffd0a0:    0x00000000    0x44434241    0x42434241    0x43424341
0xffffd0b0:    0x42434241    0x41424341    0x43414243    0x42434142
(gdb) step

```

```

Program received signal SIGSEGV, Segmentation fault.
0xf7e67ac8 in ?? () from /lib32/libc.so.6
(gdb)

```

System Command in Functions

File2.c has a command injection vulnerability within its code because of the usage of **system(cmd)** in the **do_command** function:

```
void do_command(char *cmd) {  
    system(cmd);  
}
```

What the function does is directly pass the argument inputted into the system shell without any form of input validation, which means that user input could be used to execute additional commands within the code, allowing for attackers to exploit this vulnerability for the sake of executing malicious code or shellcode to steal data or fully compromise the system.

The figure below showcases that even if the function is not called during a normal run of **file2.c**, the vulnerability is still shown clearly within GDB as by manually calling **do_command** with an input prompts the function to pass the user input directly into the system shell. An exploiter could override the return address of a function to the address of the **do_command** Function to exploit this vulnerability.

```
(gdb) break main  
Breakpoint 1 at 0x80486b0: file file2.c, line 14.  
(gdb) run data2.txt  
Starting program: /home/vde/Documents/Coursework/task2/file2.exe data2.txt  
  
Breakpoint 1, main (argc=2, argv=0xffffd1c4) at file2.c:14  
14      int select=1;  
(gdb) call do_command("echo A; echo B")  
A  
B
```

Filesize Taken Directly From External Input

Due to how **File2.c** calculates the **filesize** through the literal size of the document, the value given is then directly used to allocate the heap memory in:

```
file = malloc(filesize * sizeof(char));
```

However, this is done without any validation and the **filesize** variable comes from external input. Being used with **malloc** without any boundaries, there is no maximum size restriction nor handling for allocation failure. An attacker could exploit this to cause excessive heap memory allocation or force **Denial of Service** for the program due to inputting a **filesize** it cannot handle.

Additionally, this vulnerability also makes a vulnerable loop:

```
for (x = 0; x < filesize - 10; x++):  
    fscanf(inFile, "%c", &file[x]);
```

Where if **filesize** is under 10, the offset would cause an underflow which may cause further unexpected behaviour for the program that could be exploited by an attacker. This loop also writes into the allocated heap buffer which has its size derived from **filesize**, creating the risk of misaligned memory and potential data exploitation due to this.

```
Breakpoint 1, readFile (argv=0xffffd3b0 "smaller.txt") at file2.c:48  
48      file=malloc(filesize*sizeof(char));  
(gdb) x/24x filesize  
0x4:      Cannot access memory at address 0x4  
(gdb) x/24x file  
0xea29b700: Cannot access memory at address 0xea29b700  
(gdb) step  
50      rewind(inFile);  
(gdb) x/24x file  
0x804c6e0: 0x00000000 0x00000000 0x00000000 0x00020919  
0x804c6f0: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c700: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c710: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c720: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c730: 0x00000000 0x00000000 0x00000000 0x00000000  
(gdb) run bigger.txt  
The program being debugged has been started already.  
Start it from the beginning? (y or n) y  
Starting program: /home/vde/Documents/Coursework/task2/file2.exe bigger.txt  
File size = 57  
  
Breakpoint 1, readFile (argv=0xffffd3b1 "bigger.txt") at file2.c:48  
48      file=malloc(filesize*sizeof(char));  
(gdb) step  
50      rewind(inFile);  
(gdb) x/24x file  
0x804c6e0: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c6f0: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c700: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c710: 0x00000000 0x00000000 0x00000000 0x000208e9  
0x804c720: 0x00000000 0x00000000 0x00000000 0x00000000  
0x804c730: 0x00000000 0x00000000 0x00000000 0x00000000  
(gdb)
```

```
starting program: /home/vde/Documents/Coursework/task2/file2.exe AAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Error opening file
[Inferior 1 (process 3634) exited normally]
(gdb) █
```